

Document number: P0237R5
Date: 2017-02-06
Project: ISO JTC1/SC22/WG21: Programming Language C++
Audience: LEWG, SG14, SG6
Reply to: Vincent Reverdý <vince.rev@gmail.com>
Robert J. Brunner

Wording for fundamental bit manipulation utilities

Table of Contents

- [History and feedback](#)
 - [Introduction](#)
 - [Examples](#)
 - [Proposed wording](#)
 - [Design questions](#)
 - [Acknowledgements](#)
-

History and feedback

Original wording [February 2016 (pre-Jacksonville) → June 2016 (pre-Oulu)] ([P0237R0](#) → [P0237R1](#))

The idea of bit manipulation utilities has been originally proposed in the 2016 pre-Jacksonville mailing and has been discussed in both SG6 and LEWG in Jacksonville. The original document, [P0237R0](#) fully describes and discusses the idea of fundamental bit manipulation utilities for the C++ language. Another proposal, [P0161R0](#) had the same motivations but was focusing on bitsets. As [P0237R0](#) seemed more generic and not restrained to the scope of `std::bitset`, we decided to push forward [P0237R0](#), while still keeping in mind the functionalities and performances allowed by [P0161R0](#). The genericity and abstraction offered by [P0237R0](#) should not introduce a performance overhead (at least with the `-O2` optimization level on most compilers) when compared to [P0161R0](#) and [N3864](#).

The feedback from the Jacksonville meeting was positive: in top of their intrinsics functionalities, bit manipulation utilities have the potential to serve as a common basis for a `std::dynamic_bitset` replacement of `std::vector<bool>` and for bounded and unbounded precision arithmetic as discussed in [N4038](#).

In terms of design, the following guidance was given by LEWG:

- Restrain `std::bit_reference` to unsigned fundamental integer types?
→ [SF: 6, F: 3, N: 0, A: 0, SA :0]
- `std::bit_reference` provides no arithmetic except what the paper provides explicitly?
→ [Unanimous consent]
- In small discussion groups, introducing `std::bit_value` was considered to be good idea

In Jacksonville, the following questions were raised:

- How to deal with word endianness?
- How to provide the possibility to get a mask from the bit position?
- How to deal with infinite ranges of bits, particularly for unbounded precision arithmetic?
- What bitwise logic operators should bits provide?
- How to make `std::bit_iterator` compatible with proxy iterators presented in [P0022R1](#)?

These questions can be answered in the following way:

- The endianness is not a problem since `std::bit_iterator` is an iterator adaptor: the word-endianness is determined by the iterator provided as the template parameter
- A mask member function has been added
- Infinite ranges of bits still need discussions with SG6
- A bit should provide an equivalent of `and`, `or` and `xor`
- The compatibility with proxy iterators presented in [P0022R1](#) still need discussions

If bit utilities make it to the standard, they will be used, at least, as the basis for an implementation of a dynamic bitset, and are likely to be used as an interface to the underlying containers of words for unbounded and bounded precision arithmetic.

First update [June 2016 (pre-Oulu) → July 2016 (post-Oulu)] ([P0237R1](#) → [P0237R2](#))

The following polls were taken in LEWG:

- Should `bit_value` default-initialize to false?
→ [SF: 2, F: 5, N: 2, A: 4, SA :0] (no consensus)
- What `std::bit_pointer::difference_type` should be?

→ [intmax_t: 0, ptrdiff_t: 3, implementation defined >= ptrdiff_t: 15, int64_t: 0, unconstrained implementation defined: 7]

The following additional questions were answered in small group discussions:

- Should the classes have an independent constructor when the bit position is zero?

```
template <class UIntType>
constexpr bit_value(UIntType val) noexcept;
constexpr bit_reference(underlying_type& ref) noexcept;
constexpr bit_pointer(underlying_type* ptr) noexcept;
constexpr bit_iterator(iterator_type i);
```

→ [Not really necessary since constructors with a default position set to zero allow implementations to create independent constructors of the preceding form.]
- What should happen when the position exceeds `std::binary_digits`, and in particular should the constructors taking a position parameter be marked `noexcept`?

```
template <class UIntType>
constexpr bit_value(UIntType val, size_type pos);
constexpr bit_reference(underlying_type& ref, size_type pos);
constexpr bit_pointer(underlying_type* ptr, size_type pos);
constexpr bit_iterator(iterator_type i, size_type pos);
```

→ [It should result in an undefined behavior]
- Should the bit utility classes have an assignment operator taking the last bit of a `UIntType`?

```
template <class UIntType>
bit_value& operator=(UIntType val) noexcept;
bit_reference& operator=(underlying_type val) noexcept;
bit_pointer& operator=(underlying_type* ptr) noexcept;
bit_iterator& operator=(iterator_type i);
```

→ [Misleading and dangerous, use an assign function instead.]
- Should `std::bit_value` and `std::bit_reference` have a `set` function member taking a `bool`?

```
void set(bool b) noexcept;
```

→ [Use an assign function instead.]
- Should bit modification functions be free functions or member functions (name conflict for `set`)?

```
void set(bool b) noexcept;
void set() noexcept;
void reset() noexcept;
void flip() noexcept;
```

→ [Member functions are ok.]
- Should `std::bit_value` and `std::bit_reference` have a facet for input and output operations?
→ [Yes, but they can be added later.]
- What should happen in the streaming functions when a value that is not 0 or 1 is read as an input?
→ [The same as for integral types.]
- Should `std::bit_value`, `std::bit_reference`, `std::bit_pointer` and `std::bit_iterator` have maker functions?

```
template <class T>
constexpr bit_value make_bit_value(T val) noexcept;
template <class T>
constexpr bit_value make_bit_value(T val, typename bit_value::size_type pos);
template <class T>
constexpr bit_reference<T> make_bit_reference(T& ref) noexcept;
template <class T>
constexpr bit_reference<T> make_bit_reference(T& ref, typename bit_reference<T>::size_type pos);
template <class T>
constexpr bit_pointer<T> make_bit_pointer(T* ptr) noexcept;
template <class T>
constexpr bit_pointer<T> make_bit_pointer(T* ptr, typename bit_pointer<T>::size_type pos);
template <class T>
constexpr bit_iterator<T> make_bit_iterator(T i);
template <class T>
constexpr bit_iterator<T> make_bit_iterator(T i, typename bit_iterator<T>::size_type pos);
```

→ [Not necessary if argument deduction get accepted.]
- Should `std::bit_value` have member swap functions?

```
template <class T>
void swap(bit_reference<T> other);
void swap(bit_value& other);
```

→ [Do the same as in the current state of the standard library.]
- Should we have the whole set of swap functions?

```
template <class T, class U>
void swap(bit_reference<T> lhs, bit_reference<U> rhs) noexcept;
template <class T>
void swap(bit_reference<T> lhs, bit_value& rhs) noexcept;
template <class U>
void swap(bit_value& lhs, bit_reference<U> rhs) noexcept;
```

→ [Yes.]
- Should `std::exchange` be overloaded for `std::bit_reference`?

```
template <class T, class U = bit_value>
bit_value exchange(bit_reference<T> x, U&& val);
→ [Probably yes.]
```

- Should we have specific comparison operators for `std::bit_reference` for optimization purposes (which ones?) instead of using implicit conversion to `std::bit_value`?
→ [The choice can be left to implementers.]
- Should `std::bit_pointer` have a constructor and an assignment operator taking a `nullptr`, and should the constructor be explicit?
`explicit constexpr bit_pointer(std::nullptr_t) noexcept;`
`bit_pointer& operator=(underlying_type* ptr) noexcept;`
→ [Yes and the constructor should be implicit.]
- What should happen when a `nullptr` is provided with a non-zero position, and should a specific constructor exist for this case?
`constexpr bit_pointer(std::nullptr_t, size_type);`
→ [Undefined behavior.]
- Bikeshed the following names:
address
position
mask
underlying_type
binary_digits
→ [The small group was ok with these names.]

The following important design questions were also suggested during the meeting:

- What about concurrency and parallelism on `std::bit_reference`?
- What about infinite ranges of bits?
- What bitwise operations should be allowed on bit values and bit references?
- Should `std::bit_value` take a template parameter and provides the same interface as `std::bit_reference`?

As a consequence, the wording has been updated in the following way, compared to [P0237R1](#):

- The assignment operators from unsigned integral types have been removed:
~~`template <class UIntType>`~~
~~`bit_value& operator=(UIntType val) noexcept;`~~
~~`bit_reference& operator=(underlying_type val) noexcept;`~~
- The maker functions have been removed because of argument deduction for class templates [P0091R2](#). They may be reintroduced later if argument deduction for class templates do not make it into the standard:
~~`template <class T>`~~
~~`constexpr bit_value make_bit_value(T val) noexcept;`~~
~~`template <class T>`~~
~~`constexpr bit_value make_bit_value(T val, typename bit_value::size_type pos);`~~
~~`template <class T>`~~
~~`constexpr bit_reference<T> make_bit_reference(T& ref) noexcept;`~~
~~`template <class T>`~~
~~`constexpr bit_reference<T> make_bit_reference(T& ref, typename bit_reference<T>::size_type pos);`~~
~~`template <class T>`~~
~~`constexpr bit_pointer<T> make_bit_pointer(T* ptr) noexcept;`~~
~~`template <class T>`~~
~~`constexpr bit_pointer<T> make_bit_pointer(T* ptr, typename bit_pointer<T>::size_type pos);`~~
~~`template <class T>`~~
~~`constexpr bit_iterator<T> make_bit_iterator(T i);`~~
~~`template <class T>`~~
~~`constexpr bit_iterator<T> make_bit_iterator(T i, typename bit_iterator<T>::size_type pos);`~~
- The assignment operators with an implicit zero position have been removed because they were considered misleading:
~~`template <class UIntType>`~~
~~`bit_value& operator=(UIntType val) noexcept;`~~
~~`bit_reference& operator=(underlying_type val) noexcept;`~~
~~`bit_pointer& operator=(underlying_type* ptr) noexcept;`~~
~~`bit_iterator& operator=(iterator_type i);`~~

The future revision of this proposal will include `assign` functions, `swap` members for `std::bit_value`, and bitwise operators. Several alternative designs for `std::bit_value` with a template parameter will also be included.

Second update [July 2016 (post-Oulu) → October 2016 (pre-Issaquah)] ([P0237R2](#) → [P0237R3](#))

According to the feedback received in Oulu, the following elements have been added or removed:

- Assign member functions have been added to both `std::bit_value` and `std::bit_reference`:
`template <class UIntType>`
`void assign(UIntType val) noexcept;`
`template <class UIntType>`
`void assign(UIntType val, size_type pos);`
`void assign(underlying_type val) noexcept;`
`void assign(underlying_type val, size_type pos);`

- Bitwise operators have been added to both `std::bit_value` and `std::bit_reference`:
`bit_value& operator&=(bit_value other) noexcept;`
`bit_value& operator|=(bit_value other) noexcept;`
`bit_value& operator^=(bit_value other) noexcept;`
`constexpr bit_value operator&(bit_value lhs, bit_value rhs) noexcept;`
`constexpr bit_value operator|(bit_value lhs, bit_value rhs) noexcept;`
`constexpr bit_value operator^(bit_value lhs, bit_value rhs) noexcept;`
`bit_reference& operator&=(bit_value other) noexcept;`
`bit_reference& operator|=(bit_value other) noexcept;`
`bit_reference& operator^=(bit_value other) noexcept;`
- Swap members have been added to `std::bit_value` to match the interface of `std::bit_reference`:
`void swap(bit_value& other);`
`template <class T>`
`void swap(bit_reference<T> other);`

- Constructors of `std::bit_pointer` from `nullptr` have been updated:

```
explicit constexpr bit_pointer(std::nullptr_t) noexcept;
constexpr bit_pointer(std::nullptr_t, size_type);
```

On top of it, the two following constants were added:

- `static constexpr bit_value zero_bit(0U);`
`static constexpr bit_value one_bit(1U);`

Third update [October 2016 (pre-Issaquah) → November 2016 (post-Issaquah)] ([P0237R3](#) → [P0237R4](#))

At the Issaquah meeting, a full-room LEWG discussion on bit utilities has clarified several design questions. In particular, LEWG provided the following feedback:

- Should `std::binary_digits_v` be added to the design?
`template <class T>`
`constexpr std::size_t binary_digits_v = binary_digits<T>::value;`
→ [Yes, for consistency.] (consensus)
- When the position exceeds `binary_digits<T>::value`, it leads to undefined behavior. In that context, can the constructor taking a position parameter be marked `noexcept`?
`constexpr bit_reference(underlying_type& ref, size_type pos);`
→ [No.] (consensus)
- Should the assign member functions return a void or a reference to `*this`? Note: standard containers, `std::function` and `std::error_code` have an assign member returning void, while `std::string`, `std::regex` and `std::filesystem::path` have an assign member returning `*this`.
`void assign(underlying_type val) noexcept;`
`void assign(underlying_type val, size_type pos);`
→ [Returning `*this` is better and helps operation chaining.] (consensus)
- Should the set function member have an overload taking a `bool` as a parameter? Note: `std::bitset` has this overload.
`void set(bool b) noexcept;`
`void set() noexcept;`
→ [SF: 4, F: 11, N: 3, A: 1, SA :1] (Explanation for SA: The canonical operations on bitset are set, reset and flip. This is a fundamental characteristics.)
- Should `std::exchange` be overloaded for `std::bit_reference`?
`template <class T, class U = bit_value>`
`bit_value exchange(bit_reference<T> x, U&& val);`
→ [Lack of interest for this overload, at least for now. Can be dropped from the proposal.] (consensus)
- Bikeshedding bit constants names:
`zero_bit, one_bit` → [F: 9, A: 4]
`bit_zero, bit_one` → [F: 8, A: 4]
`bit_0, bit_1` → [F: 4, A: 9]
`bit0, bit1` → [F: 5, A: 10]
`// Constants`
`static constexpr bit_value zero_bit(0U);`
`static constexpr bit_value one_bit(1U);`
→ [(`zero_bit, one_bit`) OR (`bit_zero, bit_one`)]
- How should input and output be managed? Is a new facet necessary?
`// Stream functions`
`template <class CharT, class Traits>`
`std::basic_ostream<CharT, Traits>& operator<<(std::basic_ostream<CharT, Traits>& os, bit_value x);`
`template <class CharT, class Traits>`
`std::basic_istream<CharT, Traits>& operator>>(std::basic_istream<CharT, Traits>& is, bit_value& x);`
`template <class CharT, class Traits, class T>`
`std::basic_ostream<CharT, Traits>& operator<<(std::basic_ostream<CharT, Traits>& os, bit_reference<T> x);`
`template <class CharT, class Traits, class T>`
`std::basic_istream<CharT, Traits>& operator>>(std::basic_istream<CharT, Traits>& is, bit_reference<T>& x);`
→ [No new facet necessary, the numeric facet can be used.] (consensus)
- Should `bit_value` becomes a class template to allow implicit conversion of `bit_value&` to `bit_reference` and the implicit conversion of `bit_value*` to `bit_pointer`?

```
class bit_value;
→ [No. Most proxy iterators will be confronted to this problem, and the range proposal provides a mechanism to solve it, using a common_reference helper. For now, LEWG is not pushing in the direction of making bit_value a class template. As a technical remark, when it comes to cv-qualified template types, the standard does not take into account conversions from and to volatile versions.]
```

Additionally, the following question was raised during informal discussions:

- Should `bit_reference::operator=` returns a `bit_reference` OR a `bit_reference&`?
`bit_reference& operator=(const bit_reference& other) noexcept;`
`template <class T>`
`bit_reference& operator=(const bit_reference<T>& other) noexcept;`
`bit_reference& operator=(bit_value val) noexcept;`

According to the feedback, the proposal has been updated in the following way:

- `std::binary_digits_v` has been added to the design:
`template <class T>`
`constexpr std::size_t binary_digits_v = binary_digits<T>::value;`
- The `assign` member of `std::bit_value` and `std::bit_reference` has been modified to return `*this`:
`template <class UIntType>`
`void bit_value& assign(UIntType val) noexcept;`
`template <class UIntType>`
`void bit_value& assign(UIntType val, size_type pos);`
`void bit_reference& assign(underlying_type val) noexcept;`
`void bit_reference& assign(underlying_type val, size_type pos);`
- The `std::exchange` specialization has been removed because of the current lack of expression of interest:
`template <class T, class U = bit_value>`
`bit_value exchange(bit_reference<T> x, U&& val);`
- The wording of the input `operator>>` and output `operator<<` has been modified to match their counterparts on `std::bitset`.

Fourth update [November 2016 (post-Issaquah) → February 2017 (pre-Kona)] ([P0237R4](#) → [P0237R5](#))

Given the feedback provided during previous meetings, this document is a minor update focusing on the wording.

Introduction

This paper proposes a wording for fundamental bit utilities: `std::bit_value`, `std::bit_reference`, `std::bit_pointer` and `std::bit_iterator`. An in-depth discussion of the motivations and an in-depth exploration of the design space can be found in [P0237R0](#). In short, this paper proposes a set of 4 main utility classes to serve as bit abstractions in order to offer a common and standardized interface for libraries and programs that require bit manipulations. These bit utilities both emulate bits that are easy to use for users, and provide an interface to access the underlying words to implement efficient low-level algorithms.

Examples

An implementation of the fundamental bit utilities is available at <https://github.com/vreverd/bit>. Before presenting a wording, we illustrate some of the functionalities of the library.

First, `std::bit_value` and `std::bit_reference`:

```
// First, we create an integer
using uint_t = unsigned long long int;
uint_t intval = 42; // 101010

// Then we create aligned bit values and a bit references on this integer
std::bit_value bval0(intval); // Creates a bit value from the bit at position 0 of intval
std::bit_reference<uint_t> bref0(intval); // Creates a bit reference from the bit at position 0 of intval

// And unaligned bit values and a bit references on this integer
std::bit_value bval5(intval, 5); // Creates a bit value from the bit at position 5 of intval
std::bit_reference<uint_t> bref5(intval, 5); // Creates a bit reference from the bit at position 5 of intval

// Display them
std::cout<<bval0<<bref0<<bval5<<bref5<<std::endl; // Prints 0011

// Change their values conditionnally
if (static_cast<bool>(bval5)) {
    bval0.flip(); // Flips the bit without affecting the integer
    bref5.reset(); // Resets the bit to zero and affects the integer
}
std::cout<<bval0<<bref0<<bval5<<bref5<<std::endl; // Prints 1010

// Prints the location and the corresponding mask of bit references
std::cout<<bref0.position()<<" "<<bref0.mask()<<std::endl; // Prints 0 and 1
std::cout<<bref5.position()<<" "<<bref5.mask()<<std::endl; // Prints 5 and 32
```

Then, with `std::bit_pointer`:

```
// First, we create an array of integers
```

```

using uint_t = unsigned long long int;
std::array<uint_t, 2> intarr = {42, 314};

// Then we create a bit reference and a bit pointer
std::bit_reference<uint_t> bref5(intarr[0], 5);           // Creates a bit reference from the bit at position 5 of the first element of the array
std::bit_pointer<uint_t> bptr(intarr.data());           // Creates a bit pointer from the bit at position 0 of the first element of the array

// We flip the first bit, and sets the second one to 1 with two methods
bptr->flip();                                           // Flips the bit
++bptr;                                               // Goes to the next bit
*bptr.set();                                           // Sets the bit

// Then we advance the bit pointer by more than 64 bits and display its position
bptr += 71;
std::cout<<bptr->position()<<std::endl;               // Prints 7 as the bit is now in the second element of the array

// And finally we set the bit pointer to the position of the bit reference
bptr = &bref5;

```

And finally `std::bit_iterator`, which can serve as a basis of bit algorithm:

```

// First, we create a list of integers
using uint_t = unsigned short int;
std::list<uint_t> intlst = {40, 41, 42, 43, 44};

// Then we create a pair of aligned bit iterators
auto bfirst = std::bit_iterator<typename std::list<uint_t>::iterator>(std::begin(intlst));
auto bend = std::bit_iterator<typename std::list<uint_t>::iterator>(std::end(intlst));

// Then we count the number of bits set to 1
auto result = std::count(bfirst, bend, std::bit(1));

// We take a subset of the list
auto bfirst2 = std::bit_iterator<typename std::list<uint_t>::iterator>(std::begin(intlst), 5);
auto bend2 = std::bit_iterator<typename std::list<uint_t>::iterator>(std::end(intlst) - 1, 2);

// And we reverse the subset
std::reverse(bfirst2, bend2);

```

The count algorithm can be implemented as:

```

// Counts the number of bits equal to the provided bit value
template <class InputIt>
typename bit_iterator<InputIt>::difference_type
count(
    bit_iterator<InputIt> first,
    bit_iterator<InputIt> last,
    bit_value value
)
{
    // Assertions
    _assert_range_viability(first, last);

    // Types and constants
    using underlying_type = typename bit_iterator<InputIt>::underlying_type;
    using difference_type = typename bit_iterator<InputIt>::difference_type;
    constexpr difference_type digits = binary_digits<underlying_type>::value;

    // Initialization
    difference_type result = 0;
    auto it = first.base();
    underlying_type first_value = {};
    underlying_type last_value = {};

    // Computation when bits belong to several underlying values
    if (first.base() != last.base()) {
        if (first.position() != 0) {
            first_value = *first.base() >> first.position();
            result = _popcnt(first_value);
            ++it;
        }
        for (; it != last.base(); ++it) {
            result += _popcnt(*it);
        }
        if (last.position() != 0) {
            last_value = *last.base() << (digits - last.position());
            result += _popcnt(last_value);
        }
    }
    // Computation when bits belong to the same underlying value
    } else {
        result = _popcnt(_bextr<underlying_type>(
            *first.base(),
            first.position(),
            last.position() - first.position()
        ));
    }

    // Negates when the number of zero bits is requested
    if (!static_cast<bool>(value)) {
        result = std::distance(first, last) - result;
    }
}

```

```
// Finalization
return result;
}
```

As illustrated in these examples, bit utilities act as a convenient interface between high level code that can use bit manipulation through bit iterators, and low level algorithms that can call dedicated instruction sets and compiler intrinsics.

More information as well as more examples can be found in the presentation "[From Numerical Cosmology to Efficient Bit Abstractions for the Standard Library](#)" that was given during [CppCon](#) 2016.

Proposed Wording

Header `<bit>`

Add the following header to the standard:

```
<bit>
```

Helper struct `std::binary_digits`

Add to the `<bit>` synopsis:

```
// Binary digits structure definition
template <class T>
struct binary_digits
: std::enable_if<
    std::is_integral<T>::value && std::is_unsigned<T>::value,
    std::integral_constant<std::size_t, std::numeric_limits<T>::digits>
>::type
{};
```

```
template <class T>
constexpr std::size_t binary_digits_v = binary_digits<T>::value;
```

Requires: `std::is_integral<UIntType>::value && std::is_unsigned<UIntType>::value`.

Remarks: This helper struct allows users to extend the behavior of bit utilities on other types (for example `__uint128_t`) through their own specializations, independently from `std::numeric_limits::digits`.

Class `std::bit_value`

Add to the `<bit>` synopsis:

```
// Bit value class definition
class bit_value
{public:

    // Types
    using size_type = std::size_t;

    // Lifecycle
    bit_value() noexcept = default;
    template <class T>
    constexpr bit_value(bit_reference<T> val) noexcept;
    template <class UIntType>
    explicit constexpr bit_value(UIntType val) noexcept;
    template <class UIntType>
    constexpr bit_value(UIntType val, size_type pos);

    // Assignment
    template <class T>
    bit_value& operator=(bit_reference<T> val) noexcept;
    template <class UIntType>
    bit_value& assign(UIntType val) noexcept;
    template <class UIntType>
    bit_value& assign(UIntType val, size_type pos);

    // Bitwise assignment
    bit_value& operator&=(bit_value other) noexcept;
    bit_value& operator|=(bit_value other) noexcept;
    bit_value& operator^=(bit_value other) noexcept;

    // Conversion
    explicit constexpr operator bool() const noexcept;

    // Swap members
    void swap(bit_value& other);
    template <class T>
```

```

void swap(bit_reference<T> other);

// Bit manipulation
void set(bool b) noexcept;
void set() noexcept;
void reset() noexcept;
void flip() noexcept;
};

// Bitwise operators
constexpr bit_value operator&(bit_value lhs, bit_value rhs) noexcept;
constexpr bit_value operator|(bit_value lhs, bit_value rhs) noexcept;
constexpr bit_value operator^(bit_value lhs, bit_value rhs) noexcept;

// Comparison operators
constexpr bool operator==(bit_value lhs, bit_value rhs) noexcept;
constexpr bool operator!=(bit_value lhs, bit_value rhs) noexcept;
constexpr bool operator<(bit_value lhs, bit_value rhs) noexcept;
constexpr bool operator<=(bit_value lhs, bit_value rhs) noexcept;
constexpr bool operator>(bit_value lhs, bit_value rhs) noexcept;
constexpr bool operator>=(bit_value lhs, bit_value rhs) noexcept;

// Stream functions
template <class CharT, class Traits>
std::basic_istream<CharT, Traits>& operator>>(
    std::basic_istream<CharT, Traits>& is,
    bit_value& x
);
template <class CharT, class Traits>
std::basic_ostream<CharT, Traits>& operator<<(
    std::basic_ostream<CharT, Traits>& os,
    bit_value x
);

// Constants
static constexpr bit_value zero_bit(0U);
static constexpr bit_value one_bit(1U);

```

Lifecycle

`bit_value()` `noexcept` = default;

Effects: Constructs the bit value without initializing its value.

template <class T> constexpr bit_value(bit_reference<T> val) noexcept;

Effects: Constructs the bit value from the value of a bit reference.

template <class UIntType> explicit constexpr bit_value(UIntType val) noexcept;

Requires: `std::binary_digits<UIntType>::value` should exist and be strictly positive.

Effects: Constructs the bit value from the bit at position 0 of `val` as in `static_cast<bool>(val & static_cast<UIntType>(1))`.

template <class UIntType> constexpr bit_value(UIntType val, size_type pos);

Requires: `std::binary_digits<UIntType>::value` should exist, be strictly positive and `pos` should verify `pos < std::binary_digits<UIntType>::value`, otherwise the behavior is undefined.

Effects: Constructs the bit value from the bit at position `pos` of `val` as in `static_cast<bool>((val >> pos) & static_cast<UIntType>(1))`.

Remarks: For unsigned integral types, `static_cast<bool>((val >> pos) & static_cast<UIntType>(1))` is equivalent to `static_cast<bool>(val & (static_cast<UIntType>(1) << pos))` for `pos < std::binary_digits<UIntType>::value`.

Assignment

template <class T> bit_value& operator=(bit_reference<T> val) noexcept;

Effects: Assigns a bit reference to the bit value.

Returns: `*this`.

template <class UIntType> bit_value& assign(UIntType val) noexcept;

Effects: Assigns the bit at position 0 of `val` as in `static_cast<bool>(val & static_cast<UIntType>(1))` to the bit value.

Returns: `*this`.

template <class UIntType> bit_value& assign(UIntType val, size_type pos);

Requires: `std::binary_digits<UIntType>::value` should exist, be strictly positive and `pos` should verify `pos < std::binary_digits<UIntType>::value`, otherwise the behavior is undefined.

Effects: Assigns the bit at position `pos` of `val` as in `static_cast<bool>((val >> pos) & static_cast<UIntType>(1))` to the bit value.

Returns: `*this`. *Remarks:* For unsigned integral types, `static_cast<bool>((val >> pos) & static_cast<UIntType>(1))` is equivalent to `static_cast<bool>(val & (static_cast<UIntType>(1) << pos))` for `pos < std::binary_digits<UIntType>::value`.

Bitwise assignment

bit_value& operator&=(bit_value other) noexcept;

bit_value& operator|=(bit_value other) noexcept;

bit_value& operator^=(bit_value other) noexcept;

Effects: Assigns a new value to the bit value by performing a bitwise operation with another bit value.

Returns: `*this`.

Conversion

```
explicit constexpr operator bool() const noexcept;
```

Effects: Explicitly converts the bit value to a boolean value.

Swap members

```
void swap(bit_value& other);
template <class T> void swap(bit_reference<T> other);
```

Effects: Swaps the value of the bit value with the value of the other bit.

Bit manipulation

```
void set(bool b) noexcept;
```

Effects: Assigns `b` to the bit value.

```
void set() noexcept;
```

Effects: Unconditionally sets the bit value to 1.

```
void reset() noexcept;
```

Effects: Unconditionally resets the bit value to 0.

```
void flip() noexcept;
```

Effects: Flips the bit value, 0 becoming 1 and 1 becoming 0.

Bitwise operators

```
constexpr bit_value operator&(bit_value lhs, bit_value rhs) noexcept;
constexpr bit_value operator|(bit_value lhs, bit_value rhs) noexcept;
constexpr bit_value operator^(bit_value lhs, bit_value rhs) noexcept;
```

Effects: Applies bitwise operators to two single bits.

Returns: The bit value resulting from the operation.

Remarks: Bit references are handled through these operators using implicit conversions to bit values.

Comparison operators

```
constexpr bool operator==(bit_value lhs, bit_value rhs) noexcept;
constexpr bool operator!=(bit_value lhs, bit_value rhs) noexcept;
constexpr bool operator<(bit_value lhs, bit_value rhs) noexcept;
constexpr bool operator<=(bit_value lhs, bit_value rhs) noexcept;
constexpr bool operator>(bit_value lhs, bit_value rhs) noexcept;
constexpr bool operator>=(bit_value lhs, bit_value rhs) noexcept;
```

Effects: Compares two bit values.

Returns: The boolean value of the comparison.

Remarks: Bit references get compared through these operators using implicit conversions to bit values.

Stream functions

```
template <class CharT, class Traits> std::basic_istream<CharT, Traits>& operator>>(std::basic_istream<CharT, Traits>& is, bit_value& x);
```

Effects: A sentry object is first constructed. If the sentry object returns true, one character is extracted from `is`. If the character is successfully extracted with no end-of-file encountered, it is compared to `is.widen('0')` and to `is.widen('1')` and a temporary `bit_value` is set accordingly. If the character is neither equal to `is.widen('0')` nor to `is.widen('1')`, the extracted character is put back into the sequence. If the extraction happened without problem, the temporary bit value is assigned to `x`, otherwise `is.setstate(ios_base::failbit)` is called (which may throw `ios_base::failure`).

Returns: The updated stream `is`.

```
template <class CharT, class Traits> std::basic_ostream<CharT, Traits>& operator<<(std::basic_ostream<CharT, Traits>& os, bit_value x);
```

Effects: Outputs the bit value to the stream.

Returns: `os << os.widen(x ? '1' : '0')`

Class template `std::bit_reference`

Add to the `<bit>` synopsis:

```
// Bit reference class definition
template <class UIntType>
class bit_reference
{public:

    // Types
    using underlying_type = UIntType;
    using size_type = std::size_t;

    // Lifecycle
    template <class T>
    constexpr bit_reference(const bit_reference<T>& other) noexcept;
    explicit constexpr bit_reference(underlying_type& ref) noexcept;
    constexpr bit_reference(underlying_type& ref, size_type pos);

    // Assignment
    bit_reference& operator=(const bit_reference& other) noexcept;
    template <class T>
    bit_reference& operator=(const bit_reference<T>& other) noexcept;
    bit_reference& operator=(bit_value val) noexcept;
    bit_reference& assign(underlying_type val) noexcept;
```

```

bit_reference& assign(underlying_type val, size_type pos);

// Bitwise assignment
bit_reference& operator&=(bit_value other) noexcept;
bit_reference& operator|=(bit_value other) noexcept;
bit_reference& operator^=(bit_value other) noexcept;

// Conversion
explicit constexpr operator bool() const noexcept;

// Access
constexpr bit_pointer<UIntType> operator&() const noexcept;

// Swap members
template <class T>
void swap(bit_reference<T> other);
void swap(bit_value& other);

// Bit manipulation
void set(bool b) noexcept;
void set() noexcept;
void reset() noexcept;
void flip() noexcept;

// Underlying details
constexpr underlying_type* address() const noexcept;
constexpr size_type position() const noexcept;
constexpr underlying_type mask() const noexcept;
};

// Swap
template <class T, class U>
void swap(bit_reference<T> lhs, bit_reference<U> rhs) noexcept;
template <class T>
void swap(bit_reference<T> lhs, bit_value& rhs) noexcept;
template <class U>
void swap(bit_value& lhs, bit_reference<U> rhs) noexcept;

// Stream functions
template <class CharT, class Traits, class T>
std::basic_istream<CharT, Traits>& operator>>(
    std::basic_istream<CharT, Traits>& is,
    bit_reference<T>& x
);
template <class CharT, class Traits, class T>
std::basic_ostream<CharT, Traits>& operator<<(
    std::basic_ostream<CharT, Traits>& os,
    bit_reference<T> x
);

```

Requires: `std::binary_digits<UIntType>::value` should exist and be strictly positive.

Lifecycle

```
template <class T> constexpr bit_reference(const bit_reference<T>& other) noexcept;
```

Effects: Constructs the bit reference from a bit reference on another type τ .

Remarks: The main usage of this constructor is when τ is the same as `UIntType`, but differently cv-qualified.

```
explicit constexpr bit_reference(underlying_type& ref) noexcept;
```

Effects: Constructs the bit reference by referencing the bit at position 0 of `ref` as in `static_cast<bool>(ref & static_cast<UIntType>(1))`.

```
constexpr bit_reference(underlying_type& ref, size_type pos);
```

Requires: `pos` should verify `pos < std::binary_digits<UIntType>::value`, otherwise the behavior is undefined.

Effects: Constructs the bit reference by referencing the bit at position `pos` of `ref` as in `static_cast<bool>((ref >> pos) & static_cast<UIntType>(1))`.

Remarks: For unsigned integral types, `static_cast<bool>((ref >> pos) & static_cast<UIntType>(1))` is equivalent to `static_cast<bool>(ref & (static_cast<UIntType>(1) << pos))` for `pos < std::binary_digits<UIntType>::value`.

Assignment

```
bit_reference& operator=(const bit_reference& other) noexcept;
```

Effects: Copy assigns from the value of the bit referenced in `other`.

Returns: `*this`.

Remarks: The behavior is different than what a default copy assignment would be: the value of the `other` bit is copied, not the reference itself.

```
template <class T> bit_reference& operator=(const bit_reference<T>& other) noexcept;
```

Effects: Copy assigns from the value of the bit referenced in `other`.

Returns: `*this`. **Remarks:** The main usage of this assignment operator is when τ is the same as `UIntType`, but differently cv-qualified.

```
bit_reference& operator=(bit_value val) noexcept;
```

Effects: Assigns the value of the bit `val` to the referenced bit.

Returns: `*this`.

```
bit_reference& assign(underlying_type val) noexcept;
```

Effects: Assigns the bit at position 0 of `val` as in `static_cast<bool>(val & static_cast<UIntType>(1))` to the referenced bit.

Returns: `*this`.

```
bit_reference& assign(underlying_type val, size_type pos);
```

Requires: pos should verify pos < std::binary_digits<UIntType>::value, otherwise the behavior is undefined.

Effects: Assigns the bit at position pos of ref as in static_cast<bool>((ref >> pos) & static_cast<UIntType>(1)) to the bit reference.

Returns: *this. *Remarks:* For unsigned integral types, static_cast<bool>((ref >> pos) & static_cast<UIntType>(1)) is equivalent to static_cast<bool>(ref & (static_cast<UIntType>(1) << pos)) for pos < std::binary_digits<UIntType>::value.

Bitwise assignment

```
bit_reference& operator&=(bit_value other) noexcept;
bit_reference& operator|=(bit_value other) noexcept;
bit_reference& operator^=(bit_value other) noexcept;
```

Effects: Assigns a new value to the bit reference by performing a bitwise operation with a bit value.

Returns: *this.

Conversion

```
explicit constexpr operator bool() const noexcept;
```

Effects: Explicitly converts the bit value to a boolean value.

Access

```
constexpr bit_pointer<UIntType> operator&() const noexcept;
```

Returns: A bit pointer pointing to the referenced bit.

Swap members

```
template <class T> void swap(bit_reference<T> other);
void swap(bit_value& other);
```

Effects: Swaps the value of the referenced bit with the value of the other bit.

Bit manipulation

```
void set(bool b) noexcept;
```

Effects: Assigns b to the referenced bit.

```
void set() noexcept;
```

Effects: Unconditionally sets the referenced bit to 1.

```
void reset() noexcept;
```

Effects: Unconditionally resets the referenced bit to 0.

```
void flip() noexcept;
```

Effects: Flips the referenced bit, 0 becoming 1 and 1 becoming 0.

Underlying details

```
constexpr underlying_type* address() const noexcept;
```

Returns: A pointer to the object in which the referenced bit is.

```
constexpr size_type position() const noexcept;
```

Returns: The position of the referenced bit in the underlying object.

Remarks: The position verifies pos >= 0 and pos < std::binary_digits<UIntType>::value.

```
constexpr underlying_type mask() const noexcept;
```

Returns: A mask of underlying_type in which the bit at position pos is set to 1. The mask is equivalent to static_cast<UIntType>(1) << pos.

Swap

```
template <class T, class U> void swap(bit_reference<T> lhs, bit_reference<U> rhs) noexcept;
template <class T> void swap(bit_reference<T> lhs, bit_value& rhs) noexcept;
template <class U> void swap(bit_value& lhs, bit_reference<U> rhs) noexcept;
```

Effects: Swaps the value of the passed bits.

Stream functions

```
template <class CharT, class Traits, class T> std::basic_istream<CharT, Traits>& operator>>(std::basic_istream<CharT, Traits>& is,
bit_reference<T>& x);
```

Effects: A sentry object is first constructed. If the sentry object returns true, one character is extracted from is. If the character is successfully extracted with no end-of-file encountered, it is compared to is.widen('0') and to is.widen('1') and a temporary bit_value is set accordingly. If the character is neither equal to is.widen('0') nor to is.widen('1'), the extracted character is put back into the sequence. If the extraction happened without problem, the temporary bit value is assigned to x, otherwise is.setstate(ios_base::failbit) is called (which may throw ios_base::failure).

Returns: The updated stream is.

```
template <class CharT, class Traits, class T> std::basic_ostream<CharT, Traits>& operator<<(std::basic_ostream<CharT, Traits>& os,
bit_reference<T> x);
```

Effects: Outputs the bit reference to the stream.

Returns: os << os.widen(x ? '1' : '0')

Class template std::bit_pointer

Add to the `<bit>` synopsis:

```
// Bit pointer class definition
template <class UIntType>
class bit_pointer
{public:

    // Types
    using underlying_type = UIntType;
    using size_type = std::size_t;
    using difference_type = /* Implementation defined, at least std::ptrdiff_t */;

    // Lifecycle
    bit_pointer() noexcept = default;
    template <class T>
    constexpr bit_pointer(const bit_pointer<T>& other) noexcept;
    constexpr bit_pointer(std::nullptr_t) noexcept;
    explicit constexpr bit_pointer(underlying_type* ptr) noexcept;
    constexpr bit_pointer(underlying_type* ptr, size_type pos);

    // Assignment
    bit_pointer& operator=(std::nullptr_t) noexcept;
    bit_pointer& operator=(const bit_pointer& other) noexcept;
    template <class T>
    bit_pointer& operator=(const bit_pointer<T>& other) noexcept;

    // Conversion
    explicit constexpr operator bool() const noexcept;

    // Access
    constexpr bit_reference<UIntType> operator*() const noexcept;
    constexpr bit_reference<UIntType>* operator->() const noexcept;
    constexpr bit_reference<UIntType> operator[](difference_type n) const;

    // Increment and decrement operators
    bit_pointer& operator++();
    bit_pointer& operator--();
    bit_pointer operator++(int);
    bit_pointer operator--(int);
    constexpr bit_pointer operator+(difference_type n) const;
    constexpr bit_pointer operator-(difference_type n) const;
    bit_pointer& operator+=(difference_type n);
    bit_pointer& operator-=(difference_type n);
};

// Non-member arithmetic operators
template <class T>
constexpr bit_pointer<T> operator+(
    typename bit_pointer<T>::difference_type n,
    bit_pointer<T> x
);
template <class T, class U>
constexpr typename std::common_type<
    typename bit_pointer<T>::difference_type,
    typename bit_pointer<U>::difference_type
>::type operator-(
    bit_pointer<T> lhs,
    bit_pointer<U> rhs
);

// Comparison operators
template <class T, class U>
constexpr bool operator==(bit_pointer<T> lhs, bit_pointer<U> rhs) noexcept;
template <class T, class U>
constexpr bool operator!=(bit_pointer<T> lhs, bit_pointer<U> rhs) noexcept;
template <class T, class U>
constexpr bool operator<(bit_pointer<T> lhs, bit_pointer<U> rhs) noexcept;
template <class T, class U>
constexpr bool operator<=(bit_pointer<T> lhs, bit_pointer<U> rhs) noexcept;
template <class T, class U>
constexpr bool operator>(bit_pointer<T> lhs, bit_pointer<U> rhs) noexcept;
template <class T, class U>
constexpr bool operator>=(bit_pointer<T> lhs, bit_pointer<U> rhs) noexcept;
```

Requires: `std::binary_digits<UIntType>::value` should exist and be strictly positive.

Remarks: In the class definition above, `difference_type` is implementation defined but is required to be a signed integer of at least `std::ptrdiff_t`'s size.

Lifecycle

```
bit_pointer() noexcept = default;
```

Effects: Constructs the bit pointer and leaves it uninitialized.

```
template <class T> constexpr bit_pointer(const bit_pointer<T>& other) noexcept;
```

Effects: Constructs the bit pointer from a bit pointer on another type `T`.

Remarks: The main usage of this constructor is when `T` is the same as `UIntType`, but differently cv-qualified.

```
constexpr bit_pointer(std::nullptr_t) noexcept;
```

Effects: Constructs a null bit pointer with position `0`.

```
explicit constexpr bit_pointer(underlying_type* ptr) noexcept;
```

Effects: Constructs the bit pointer by pointing to the bit at position 0 of ptr as in static_cast<bool>(*ptr & static_cast<UIntType>(1)).

```
constexpr bit_pointer(underlying_type* ptr, size_type pos);
```

Requires: pos should verify pos < std::binary_digits<UIntType>::value, otherwise the behavior is undefined.

Effects: Constructs the bit pointer by pointing to the bit at position pos of *ptr as in static_cast<bool>((*ptr >> pos) & static_cast<UIntType>(1)).

Remarks: For unsigned integral types, static_cast<bool>((*ptr >> pos) & static_cast<UIntType>(1)) is equivalent to static_cast<bool>(*ptr & (static_cast<UIntType>(1) << pos)) for pos < std::binary_digits<UIntType>::value.

Assignment

```
bit_pointer& operator=(std::nullptr_t) noexcept;
```

Effects: Sets the bit pointer to a null bit pointer with position 0.

Returns: *this.

```
bit_pointer& operator=(const bit_pointer& other) noexcept;
```

Effects: Copy assigns from the other bit pointer.

Returns: *this.

```
template <class T> bit_pointer& operator=(const bit_pointer<T>& other) noexcept;
```

Effects: Copy assigns from the other bit pointer.

Returns: *this.

Conversion

```
explicit constexpr operator bool() const noexcept;
```

Effects: Returns false if the underlying pointer is null and the position is 0, returns true otherwise.

Access

```
constexpr bit_reference<UIntType> operator*() const noexcept;
```

Returns: A bit reference referencing the pointed bit.

```
constexpr bit_reference<UIntType>* operator->() const noexcept;
```

Returns: A pointer to a bit reference referencing the pointed bit.

```
constexpr bit_reference<UIntType> operator[](difference_type n) const;
```

Returns: A bit reference referencing the n-th bit after the pointed bit.

Remarks: In that context, if pos + 1 < std::binary_digits<UIntType>::value, the next bit is the bit at pos + 1 in the same underlying object, but if pos + 1 == std::binary_digits<UIntType>::value, the next bit is the bit at position 0 of the next underlying object in memory.

Increment and decrement operators

```
bit_pointer& operator++();
bit_pointer& operator--();
bit_pointer operator++(int);
bit_pointer operator--(int);
constexpr bit_pointer operator+(difference_type n) const;
constexpr bit_pointer operator-(difference_type n) const;
bit_pointer& operator+=(difference_type n);
bit_pointer& operator-=(difference_type n);
```

Effects: Performs increment and decrement operations.

Remarks: In that context, if pos + 1 < std::binary_digits<UIntType>::value, the next bit is the bit at pos + 1 in the same underlying object, but if pos + 1 == std::binary_digits<UIntType>::value, the next bit is the bit at position 0 of the next underlying object in memory.

Non-member arithmetic operators

```
template <class T> constexpr bit_pointer<T> operator+(typename bit_pointer<T>::difference_type n, bit_pointer<T> x);
```

Returns: A bit pointer to the x + n bit.

```
template <class T, class U> constexpr typename std::common_type< typename bit_pointer<T>::difference_type, typename bit_pointer<U>::difference_type >::type operator-(bit_pointer<T> lhs, bit_pointer<U> rhs);
```

Returns: The number of bit pointer increments n so that rhs + n == lhs.

Comparison operators

```
template <class T, class U> constexpr bool operator==(bit_pointer<T> lhs, bit_pointer<U> rhs) noexcept;
template <class T, class U> constexpr bool operator!=(bit_pointer<T> lhs, bit_pointer<U> rhs) noexcept;
template <class T, class U> constexpr bool operator<(bit_pointer<T> lhs, bit_pointer<U> rhs) noexcept;
template <class T, class U> constexpr bool operator<=(bit_pointer<T> lhs, bit_pointer<U> rhs) noexcept;
template <class T, class U> constexpr bool operator>(bit_pointer<T> lhs, bit_pointer<U> rhs) noexcept;
template <class T, class U> constexpr bool operator>=(bit_pointer<T> lhs, bit_pointer<U> rhs) noexcept;
```

Effects: Compares two bit pointers by comparing the pointer to the underlying object first, and the position of the bit in that object if the two underlying pointers are equal.

Returns: The boolean value of the comparison.

Class template std::bit_iterator

Add to the <bit> synopsis:

```
// Bit iterator class definition
```

```

template <class Iterator>
class bit_iterator
{public:

    // Types
    using iterator_type = Iterator;
    using underlying_type = typename _cv_iterator_traits<Iterator>::value_type;
    using iterator_category = typename std::iterator_traits<Iterator>::iterator_category;
    using value_type = bit_value;
    using difference_type = /* Implementation defined, at least std::ptrdiff_t */;
    using pointer = bit_pointer<underlying_type>;
    using reference = bit_reference<underlying_type>;
    using size_type = std::size_t;

    // Lifecycle
    constexpr bit_iterator();
    template <class T>
    constexpr bit_iterator(const bit_iterator<T>& other);
    explicit constexpr bit_iterator(iterator_type i);
    constexpr bit_iterator(iterator_type i, size_type pos);

    // Assignment
    template <class T>
    bit_iterator& operator=(const bit_iterator<T>& other);

    // Access
    constexpr reference operator*() const noexcept;
    constexpr pointer operator->() const noexcept;
    constexpr reference operator[](difference_type n) const;

    // Increment and decrement operators
    bit_iterator& operator++();
    bit_iterator& operator--();
    bit_iterator operator++(int);
    bit_iterator operator--(int);
    constexpr bit_iterator operator+(difference_type n) const;
    constexpr bit_iterator operator-(difference_type n) const;
    bit_iterator& operator+=(difference_type n);
    bit_iterator& operator-=(difference_type n);

    // Underlying details
    constexpr iterator_type base() const;
    constexpr size_type position() const noexcept;
    constexpr underlying_type mask() const noexcept;
};

// Non-member arithmetic operators
template <class T>
constexpr bit_iterator<T> operator+(
    typename bit_iterator<T>::difference_type n,
    const bit_iterator<T>& i
);
template <class T, class U>
constexpr typename std::common_type<
    typename bit_iterator<T>::difference_type,
    typename bit_iterator<U>::difference_type
>::type operator-(
    const bit_iterator<T>& lhs,
    const bit_iterator<U>& rhs
);

// Comparison operators
template <class T, class U>
constexpr bool operator==(const bit_iterator<T>& lhs, const bit_iterator<U>& rhs);
template <class T, class U>
constexpr bool operator!=(const bit_iterator<T>& lhs, const bit_iterator<U>& rhs);
template <class T, class U>
constexpr bool operator<(const bit_iterator<T>& lhs, const bit_iterator<U>& rhs);
template <class T, class U>
constexpr bool operator<=(const bit_iterator<T>& lhs, const bit_iterator<U>& rhs);
template <class T, class U>
constexpr bool operator>(const bit_iterator<T>& lhs, const bit_iterator<U>& rhs);
template <class T, class U>
constexpr bool operator>=(const bit_iterator<T>& lhs, const bit_iterator<U>& rhs);

```

Requires: `std::binary_digits<std::iterator_traits<Iterator>::value_type>::value` should exist and be strictly positive.

Remarks: In the class definition above, `_cv_iterator_traits` is an internal helper struct (for illustration purpose only) whose `value_type` preserves cv-qualifiers, contrarily to `std::iterator_traits`. Also, `difference_type` is implementation defined but is required to be a signed integer of at least `std::ptrdiff_t`'s size.

Lifecycle

```
constexpr bit_iterator();
```

Effects: Value initializes the underlying iterator. Iterator operations applied to the resulting iterator have defined behavior if and only if the corresponding operations are defined on a value-initialized iterator of type `Iterator`.

```
template <class T> constexpr bit_iterator(const bit_iterator<T>& other);
```

Effects: Constructs the bit iterator from a bit iterator on another type `T`.

```
explicit constexpr bit_iterator(iterator_type i);
```

Effects: Constructs the bit iterator from an iterator assuming a bit position 0.

```
constexpr bit_iterator(iterator_type i, size_type pos);
```

Requires: pos should verify pos < std::binary_digits<underlying_type>::value, otherwise the behavior is undefined.

Effects: Constructs the bit iterator by pointing to the bit at position pos of *it as in static_cast<bool>((*it >> pos) & static_cast<underlying_type>(1)).

Remarks: For unsigned integral types, static_cast<bool>((*it >> pos) & static_cast<underlying_type>(1)) is equivalent to static_cast<bool>(*it & (static_cast<underlying_type>(1) << pos)) for pos < std::binary_digits<underlying_type>::value.

Assignment

```
template <class T> bit_iterator& operator=(const bit_iterator<T>& other);
```

Effects: Copy assigns from the other bit iterator.

Returns: *this.

Access

```
constexpr reference operator*() const noexcept;
```

Returns: A bit reference referencing the bit corresponding to the current bit iterator.

```
constexpr pointer operator->() const noexcept;
```

Returns: A pointer to a bit reference referencing the bit corresponding to the current bit iterator.

```
constexpr reference operator[](difference_type n) const;
```

Returns: A bit reference referencing the n-th bit after the bit corresponding to the current bit iterator.

Remarks: In that context, if pos + 1 < std::binary_digits<underlying_type>::value, the next bit is the bit at pos + 1 in the same underlying object, but if pos + 1 == std::binary_digits<underlying_type>::value, the next bit is the bit at position 0 of the next iterator.

Increment and decrement operators

```
bit_iterator& operator++();
bit_iterator& operator--();
bit_iterator operator++(int);
bit_iterator operator--(int);
constexpr bit_iterator operator+(difference_type n) const;
constexpr bit_iterator operator-(difference_type n) const;
bit_iterator& operator+=(difference_type n);
bit_iterator& operator-=(difference_type n);
```

Effects: Performs increment and decrement operations.

Remarks: In that context, if pos + 1 < std::binary_digits<underlying_type>::value, the next bit is the bit at pos + 1 in the same underlying object, but if pos + 1 == std::binary_digits<underlying_type>::value, the next bit is the bit at position 0 of the next iterator.

Underlying details

```
constexpr iterator_type base() const;
```

Returns: An iterator to the object in which the referenced bit is.

```
constexpr size_type position() const noexcept;
```

Returns: The position of the referenced bit in the underlying object.

Remarks: The position verifies pos >= 0 and pos < std::binary_digits<underlying_type>::value.

```
constexpr underlying_type mask() const noexcept;
```

Returns: A mask of underlying_type in which the bit at position pos is set to 1. The mask is equivalent to static_cast<underlying_type>(1) << pos.

Non-member arithmetic operators

```
template <class T> constexpr bit_iterator<T> operator+(typename bit_iterator<T>::difference_type n, const bit_iterator<T>& i);
```

Returns: A bit iterator to the x + n bit.

```
template <class T, class U> constexpr typename std::common_type<typename bit_iterator<T>::difference_type, typename
bit_iterator<U>::difference_type>::type operator-(const bit_iterator<T>& lhs, const bit_iterator<U>& rhs);
```

Returns: The number of bit iterator increments n so that rhs + n == lhs.

Comparison operators

```
template <class T, class U> constexpr bool operator==(const bit_iterator<T>& lhs, const bit_iterator<U>& rhs);
template <class T, class U> constexpr bool operator!=(const bit_iterator<T>& lhs, const bit_iterator<U>& rhs);
template <class T, class U> constexpr bool operator<(const bit_iterator<T>& lhs, const bit_iterator<U>& rhs);
template <class T, class U> constexpr bool operator<=(const bit_iterator<T>& lhs, const bit_iterator<U>& rhs);
template <class T, class U> constexpr bool operator>(const bit_iterator<T>& lhs, const bit_iterator<U>& rhs);
template <class T, class U> constexpr bool operator>=(const bit_iterator<T>& lhs, const bit_iterator<U>& rhs);
```

Effects: Compares two bit iterators by comparing the iterator to the underlying object first, and the position of the bit in that object if the two underlying iterators are equal.

Returns: The boolean value of the comparison.

Design questions

As most design questions have been answered over the last meetings, no major design issue seem to remain. Additional bikeshedding could be done on the following:

- `underlying_type` VS `word_type`
 - `zero_bit` VS `bit_zero` VS `bit0`
 - `one_bit` VS `bit_one` VS `bit1`
-

Acknowledgements

The authors would like to thank Tomasz Kaminski, Titus Winters, Lawrence Cowl, Howard Hinnant, Jens Maurer, Tony Van Eerd, Klemens Morgenstern, Vicente Botet Escriba, Odin Holmes and the other contributors of the ISO C++ Standard - Discussion and of the ISO C++ Standard - Future Proposals groups for their initial reviews and comments.

Vincent Reverdy and Robert J. Brunner have been supported by the National Science Foundation Grant AST-1313415. Robert J. Brunner has been supported in part by the Center for Advanced Studies at the University of Illinois.